

918. Maximum Sum Circular Subarray

Given a circular integer array `nums` of length `n`, return the maximum possible sum of a non-empty **subarray** of `nums`.

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist $i \leq k_1, k_2 \leq j$ with $k_1 \neq k_2$ and `nums[k_1] == nums[k_2]`.

Example 1:

Input: `nums = [1,-2,3,-2]`
Output: `3`
Explanation: Subarray `[3]` has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`
Output: `10`
Explanation: Subarray `[5,5]` has maximum sum $5 + 5 = 10$.

Example 3:

Input: `nums = [-3,-2,-3]`
Output: `-2`
Explanation: Subarray `[-2]` has maximum sum -2.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 3 \times 10^4$
- $-3 \times 10^4 \leq \text{nums}[i] \leq 3 \times 10^4$

Code

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int totalSum = 0, maxSum = nums[0], minSum = nums[0];
        int current = 0, currentMin = 0;
        for (int num : nums) {
            totalSum += num;
            current = Math.max(current, current + num);
            maxSum = Math.max(maxSum, current);
            currentMin = Math.min(currentMin, current + num);
            minSum = Math.min(minSum, currentMin);
        }
        return (maxSum > 0) ? Math.max(maxSum, totalSum - minSum) : maxSum;
    }
}
```

Testcase

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

`[1,-2,3,-2]`

Output

`3`

Expected

`3`

8. String to Integer (atoi)

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

- Whitespace:** Ignore any leading whitespace (`" "`).
- Sign:** Determine the sign by checking if the next character is `'+'` or `'-'`, assuming positivity if neither present.
- Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
- Rounding:** If the integer is out of the 32-bit signed integer range `[-231, 231 - 1]`, then round the integer to remain in the range. Specifically, integers less than `-231` should be rounded to `-231`, and integers greater than `231 - 1` should be rounded to `231 - 1`.

Return the integer as the final result.

Example 1:

Input: `s = "42"`
Output: `42`
Explanation:
The underlined characters are what is read in and the caret is the current reader position.
Step 1: `"42"` (no characters read because there is no leading whitespace)
Step 2: `"42"` (no characters read because there is neither a `'+'` nor `'-'`)
Step 3: `"42"` (`"42"` is read in)

Example 2:

Input: `s = "-42"`

Constraints:

- $1 \leq s.length \leq 10^4$
- `s` consists of English letters, digits, spaces, and the symbols `'+'` and `'-'`.

Code

```
class Solution {
    public int myAtoi(String s) {
        int digit = curr == '0';
        // Stop at check for 32-bit signed integer overflow/underflow
        if (result > Integer.MAX_VALUE / 10 ||
            (result == Integer.MAX_VALUE / 10 && digit > Integer.MAX_VALUE % 10)) {
            return Integer.MAX_VALUE;
        }
        result = result * 10 + digit;
        if (digit == '-') {
            return -result;
        }
        return result;
    }
}
```

Testcase

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Case 4

Case 5

Input

`s = "-42"`

Output

`-42`

Expected

`-42`

Alternating Characters

You are given a string consisting of characters `A` and `B`. Only two test cases are changed in this string such that there are no matching adjacent characters. In other words, you are allowed to change one or more characters in the string.

Return the minimum number of required changes.

Example:

Input: `s = "AABAB"`
Output: `1`
Explanation: The string is `"AABAB"`. The character at index 1 is `A` and the character at index 2 is `A`. We can change the character at index 1 to `B` to make the string `"BABAB"`. The string `"BABAB"` has the minimum number of changes required.

Constraints:

- $1 \leq s.length \leq 10^4$
- `s` consists of characters `A` and `B`.

Code

```
import java.util.*;

public class Solution {
    public static int alternatingCharacters(String s) {
        int count = 0;
        for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Number of test cases
        int q = sc.nextInt();
        // Process each string
        for (int i = 0; i < q; i++) {
            String s = sc.next();
            int answer = alternatingCharacters(s);
            System.out.println(answer);
        }
        sc.close();
    }
}
```

Testcase

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

`s = "AABAB"`

Output

`1`

Expected

`1`

3. Longest Substring Without Repeating Characters

Given a string `s`, find the length of the **longest substring** without repeating characters.

Example 1:

Input: `s = "abcabcbb"`
Output: `3`
Explanation: The answer is `"abc"`, with the length of 3.

Example 2:

Input: `s = "bbbbb"`
Output: `1`
Explanation: The answer is `"b"`, with the length of 1.

Example 3:

Input: `s = "pwwkpw"`
Output: `4`
Explanation: The answer is `"wkwk"`, with the length of 4. The answer must be a substring, `"pwkw"` is not a substring and `pwkpw` is not a substring.

Constraints:

- $0 \leq s.length \leq 5 \times 10^4$
- `s` consists of English letters, digits, symbols and spaces.

Code

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int left = 0, right = 0, maxLen = 0;
        Map<Character, Integer> map = new HashMap<>();
        while (right < s.length()) {
            if (map.containsKey(s.charAt(right))) {
                left = map.get(s.charAt(right)) + 1;
            }
            map.put(s.charAt(right), right);
            maxLen = Math.max(maxLen, right - left + 1);
            right++;
        }
        return maxLen;
    }
}
```

Testcase

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

`s = "abcabcbb"`

Output

`3`

Expected

`3`

Problem List < > 25 Submit

Description Editorial Solutions Submissions

890. Find and Replace Pattern

Given a list of strings `words` and a string `pattern`, return a list of `words[i]` that match `pattern`. You may return the answer in any order.

A word matches the pattern if there exists a permutation of letters `p` so that after replacing every letter `x` in the pattern with `p(x)`, we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

```
Input: words = ["abc","bad","baa","aag","aba","ccc"], pattern = "abb"
Output: ["baa","aag"]
Explanation: "baa" matches the pattern because there is a permutation {a -> b, b -> a, c -> c}.
"ccc" does not match the pattern because {a -> c, b -> c, c -> c} is not a permutation, since a and b map to the same letter.
```

Example 2:

```
Input: words = ["a","b","c"], pattern = "aa"
Output: ["a","b","c"]
```

Constraints:

- `1 <= pattern.length <= 100`
- `1 <= words.length <= 100`
- `words[i].length == pattern.length`
- `words[i]` and `pattern[i]` consist of lowercase English letters.

Discover more

48 26 0 0

Code

```
class Solution {
public:
    vector<string> findAndReplacePattern(string pattern, vector<string>& words) {
        vector<string> result;
        for (string word : words) {
            if (isIsomorphic(word, pattern)) {
                result.push_back(word);
            }
        }
        return result;
    }
};
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

```
words = ["abc","bad","baa","aag","aba","ccc"]
pattern = "abb"
```

Output

```
["baa","aag"]
```

Expected

```
["baa","aag"]
```

leetcode.com/problems/string-matching-in-an-array/

Problem List < > 25 Submit

Description Editorial Solutions Submissions

1406. String Matching in an Array

Given an array of strings `words`, return all strings in `words` that are a **substring** of another word. You can return the answer in any order.

Example 1:

```
Input: words = ["mass","as","hero","superhero"]
Output: ["as","hero"]
Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".
["hero","as"] is also a valid answer.
```

Example 2:

```
Input: words = ["leetcode","et","code"]
Output: ["et","code"]
Explanation: "et" and "code" are substrings of "leetcode".
```

Example 3:

```
Input: words = ["a","b","c","d","e","f"]
Output: []
Explanation: No string in words is substring of another string.
```

Constraints:

- `1 <= words.length <= 100`
- `1 <= words[i].length <= 20`
- `words[i]` contains only lowercase English letters.
- All the strings of `words` are unique.

Discover more

136 73 0 0

Code

```
class Solution {
public:
    vector<string> stringMatching(vector<string>& words) {
        vector<string> result;
        for (int i = 0; i < words.size(); i++) {
            for (int j = 0; j < words.size(); j++) {
                if (i != j && words[i].find(words[j]) != string::npos) {
                    result.push_back(words[i]);
                }
            }
        }
        return result;
    }
};
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

```
words = ["mass","as","hero","superhero"]
```

Output

```
["as","hero"]
```

Expected

```
["as","hero"]
```